

Topic 4: Data Cleaning and Preparation

Removal of outliers

Igor Razgon

igor.razgon@durham.ac.uk

Removal of outliers

The task of removal of outliers

- Let *df* be a *DataFrame* instance, *col* be a column of *df*.
- A value occurring in *col* is an *outlier* if it occurs *rarely* in *col*.
- **The task of removal of outliers:** remove the rows of the *df* whose values at *col* are outliers.

What is a 'rare occurrence'?

- In order to design an algorithm for identification of outliers, we need to provide a precise specification of a *rare occurrence*.
- We will consider the following three interpretations.
 - **Threshold limit:** We set some threshold limit, say 3, and all the values that occur less than 3 times are considered outliers.
 - **Threshold limit for clusters:** We classify all the values into several classes and again set a limit, say 3. A value is considered an outlier if its class occurs less than 3 times.
 - **Quartiles based:** We apply a standard statistical methodology involving quartiles and interquartile range.

The Pandas *value_counts()* function

- Let *sr* be a *Series* instance.
- *sr.value_counts()* returns a series indexed by the values of *sr*.
- Each value of *sr* is mapped to its frequency (the number of times it occurs in *sr*).
- The values are ordered by the decreasing order of their occurrences.

Application of `value_counts()` to a *DataFrame* column

```
testdict={  
    'key1': ['a','b','a','a','a','b','a','b'],  
    'key2': ['c','c','c','c','d','d','d','d'],  
    'num1': [1,1,3,3,3,6,7,8],  
    'num2': [40,35,30,25,20,15,10,5]  
}
```

```
freq_test=pd.DataFrame(testdict)
```

#Computing the number of occurrences of each value of 'num1'

#Note that value_counts is a function of Series not of DataFrame

#Note also that the input is indexed by the values of 'num1'

```
valcounts=freq_test['num1'].value_counts()
```

valcounts

```
3      3  
1      2  
6      1  
7      1  
8      1
```

```
Name: num1, dtype: int64
```

Identification and removal of outlier values

- The output of `value_counts()` provides all the information we need for identification of outlier values.
- They are simply the values whose counts are *below the threshold*.
- We next filter these values out leaving only those that are not outliers.

#Removal of those values that occur only once

```
valreduced=valcounts[valcounts>1]
```

```
valreduced
```

```
3      3
1      2
Name: num1, dtype: int64
```

Pandas function *isin()*

- Let *sr1* and *sr2* be series instances.
- *sr1.isin(sr2)* returns a Boolean *Series* instance with the same index as *sr1*.
- The values of *sr1* corresponding to *True* are exactly those that occur in *sr2*.
- The function *isin()* is the last piece we need for removal of rare occurrences.

The last step of filtering rare occurrences for column *num1*

freq_test

#Logical mask for frequently occurring values

freq_mask=freq_test['num1'].isin(valreduced.index)

freq_mask

#Removal of outliers from th initial dataframe

freq_test_rm=freq_test[freq_mask]

freq_test_rm

	key1	key2	num1	num2
0	a	c	1	40
1	b	c	1	35
2	a	c	3	30
3	a	c	3	25
4	a	d	3	20

```
0      True
1      True
2      True
3      True
4      True
5     False
6     False
7     False
Name: num1, dtype: bool
```

	key1	key2	num1	num2
0	a	c	1	40
1	b	c	1	35
2	a	c	3	30
3	a	c	3	25
4	a	d	3	20
5	b	d	6	15
6	a	d	7	10
7	b	d	8	5

Removal of values that belong to rarely appearing clusters

- Individual values are often rare.
- However, after we partition the values into classes, we identify some classes as occurring often and some classes as occurring rare.
- We are going to discuss removal of rows whose corresponding values belong to these rarely occurring classes.

Example of outlier classes

- A class of 40 students had their exam.
- Every individual mark occurs at most twice (but it does not make sense to consider all of them outliers).
- However, if replace each mark with its honour class, we notice that the 'Fail' and 'Third' classes occur only once, while the remaining classes occur at least three times.
- Thus, the marks falling into the 'Fail' and 'Third' categories are outliers.

Some analogy with random variables over continuous domains

- Suppose a random variable takes real values.
- Then the probability of each individual value is 1 .
- In other words, each individual value is an outlier.
- Therefore, it makes much more sense to measure probabilities of intervals of values.

The idea of the cluster removal

- Suppose that we want to remove the values of *col* in a *DataFrame* instance *df* that belongs to a rare cluster.
- Let *mf* be the function mapping the values of *col* to the clusters.
- The *val_counts()* with the subsequent filtering are applied to *df[col].map(mf)* rather than to *df[col]* directly.
- One to implement this is to create a new column obtained by mapping of *col* by *mf* and then to use the approach of removal of rare occurrences that we have just considered.
- Examples of column mapping and creating a new column have been considered during the classes of Week 7.
- Please complete the details yourself (there will be a corresponding example in the next practical).

Quartiles based removal of outliers

- We will define the notions of quartiles and interquartile range.
- Using these notions, we will identify *upper* and *lower bounds* for the given dataset, so that all the values *beyond these bounds* will be considered *outliers*.

Definition of quartiles

- Quartiles are defined over a sequence S of numbers (data points)
- $Q1$ or the **first quartile**: an element x of S is smaller than $Q1$ if and only if x belongs to the first 25% of S .
- $Q2$ or the **second quartile**: an element x of S is smaller than $Q1$ if and only if x belongs to the first 50% of S .
- $Q3$ or the **third quartile**: an element x of S is smaller than $Q1$ if and only if x belongs to the first 75% of S .

Computation of quartiles

- The definition of quartiles does not specify an algorithm for their computation.
- Indeed, there is no single agreed method for quartile computation:
<https://en.wikipedia.org/wiki/Quartile>
- We will see how quartiles are computed in Python and consider several examples of their computation.

Computation of quartiles in Python

```
qs=pd.Series([1,2,3,4,5,6,7,8,9,10,11,12,13])
```

```
quartiles=qs.quantile([0.25,0.5,0.75])
```

#Quartiles belonging to data points

```
quartiles      0.25      4.0  
              0.50      7.0  
              0.75     10.0  
dtype: float64
```

```
qs=pd.Series([1,2,3,4,5,6,7,8,9,10,11,12])
```

```
quartiles=qs.quantile([0.25,0.5,0.75])
```

#Quartiles not belonging to data points

```
quartiles      0.25      3.75  
              0.50      6.50  
              0.75      9.25  
dtype: float64
```

Interquartile range and the outlier bounds

- The **interquartile range** IQR is just $Q_3 - Q_1$.
- For computation of outliers we define
 - **Lower bound** $lb = Q_1 - 1.5 * IQR$
 - **Upper bound** $ub = Q_3 + 1.5 * IQR$
- *Outliers* are data points that are either *smaller than lb* or *greater than ub* .

Applicability of the statistical computation of outliers

- Suppose that Q_3 is not much larger than Q_1 , hence IQR is small.
- Then the methodology effectively cuts small values on the left and large values on the right of the dataset.

Function for computation of lower and upper bounds

```
def OutliersBound(sr):  
    quartiles=sr.quantile([0.25,0.5,0.75])  
    iqr=quartiles[0.75]-quartiles[0.25]  
    lb=quartiles[0.25]-1.5*iqr  
    ub=quartiles[0.75]+1.5*iqr  
    return (lb,ub)
```

Interquartile-based removal of outliers from a data frame

```
pdsalaries = pd.read_csv('salaryoutliers.csv')
pdsalaries

#Bounds for interquartile range filtering
bounds=OutliersBound(pdsalaries['salary'])
bounds

#Mask for filtering salaries that are too low
lb patt=pdsalaries['salary']>=bounds[0]

#Mask for filtering salaries that are too high
ub patt=pdsalaries['salary']<=bounds[1]

#Connecting the masks through logical_and
salaries_cleaned=pdsalaries[np.logical_and(lb patt,ub patt)]
salaries_cleaned
```

(1500.0, 7900.0)

	id	salary
0	21	5000
1	22	5088
2	38	4900
3	33	900
4	44	100
5	45	5400
6	77	5800
7	115	20000

	id	salary
0	21	5000
1	22	5088
2	38	4900
5	45	5400
6	77	5800

Which method is preferable

- Depends on the application.
- Filtering with the limit looks reasonable for analysis of record of employees or student (small number of vacation days, exceptional marks, etc.)
- For applications related to Natural Sciences (e.g. time series with temperature observations), the statistical method looks preferable.